```
  const optimalClusters = Math.max(1, Math.min(Math.ceil(
Math.sqrt(numChunks / 2)), numChunks - 1));

  console.log('Number of document chunks:', numChunks);
  console.log('Optimal number of clusters:', optimalClusters);

  const kmeansResult =
kmeans(flattenedEmbeddings, optimalClusters);
  const mergedChunks =
mergeSimilarClusters(documentChunks, kmeansResult.clusters,
similarityThreshold);

  return mergedChunks;
}
```

Calculates the optimal number of clusters using the square root of half the number of chunks, bounded between 1 and numChunks - 1

Applies k-means clustering to the document embeddings using the calculated optimal number of clusters

Merges similar chunks within each cluster based on the similarity threshold to create coherent document sections

First, the input document is split into manageable chunks using `RecursiveCharacterTextSplitter` with a chunk size of 10,000 characters and an overlap of 250 characters. The `clusterDocuments` function then employs the Hugging Face pipeline with the `Xenova/all-MiniLM-L6-v2` model for feature extraction, converting each text chunk into a dense vector representation. The algorithm next determines the optimal number of clusters, using a heuristic formula based on the document size.

Finally, `clusterDocuments` implements a similarity threshold to merge clusters that are semantically related. This merging step is handled by the helper function `mergeSimilarClusters`, which combines chunks from the same cluster.

Implementation tips

While this code demonstrates a more complex document-clustering approach using k-means, users don't need to deeply understand all the internal mechanics. The key aspects are

- The algorithm provided to compute *K* is a practical heuristic formula that takes the square root of half the number of chunks. The formula isn't magical—it's a practical compromise that works well for document clustering while scaling reasonably with document size.
- The embedding and clustering process is abstracted away behind easy-to-use functions.
- The similarity threshold (0.8 by default) is the main parameter users might want to adjust where higher values create tighter, more similar clusters while lower values allow more diversity.

NOTE While using techniques like sentence splitting and k-means clustering can help in organizing and summarizing large documents, there will inevitably be some information loss. This is particularly relevant to how the k-means clustering algorithm is configured and tuned, as different settings can significantly affect the granularity and coherence of the resulting clusters.